DATABASE ARCHITECTURE · A DECISION FRAMEWORK · 2026

When Not to Use Postgres

A decision framework for the four walls where one engine isn't enough

One ACID engine now absorbs vectors, time-series, queues, search, and documents. Here is the consolidation case for defaulting to Postgres — and the four specific walls where I still reach for a specialist.

10 slides · first-person field notes

THE ANECDOTE · FIELD NOTES

Five datastores, one product

Across years with customers, I've watched **team after team** hit the same knot. One ran **five moving parts** to serve a single product:

- **Postgres** — the relational core
- **Redis** — the hot path
- **Elasticsearch** — search
- **A message queue** — background jobs
- **A cron service** — scheduled work

Five backup stories. Five things to monitor, patch, secure, and get paged about at 2 a.m. **We collapsed all of it into one Postgres** — fewer moving parts, and the cloud bill went down because we stopped paying for four managed services.

WHY NOW · THE MEASURED DEFAULT

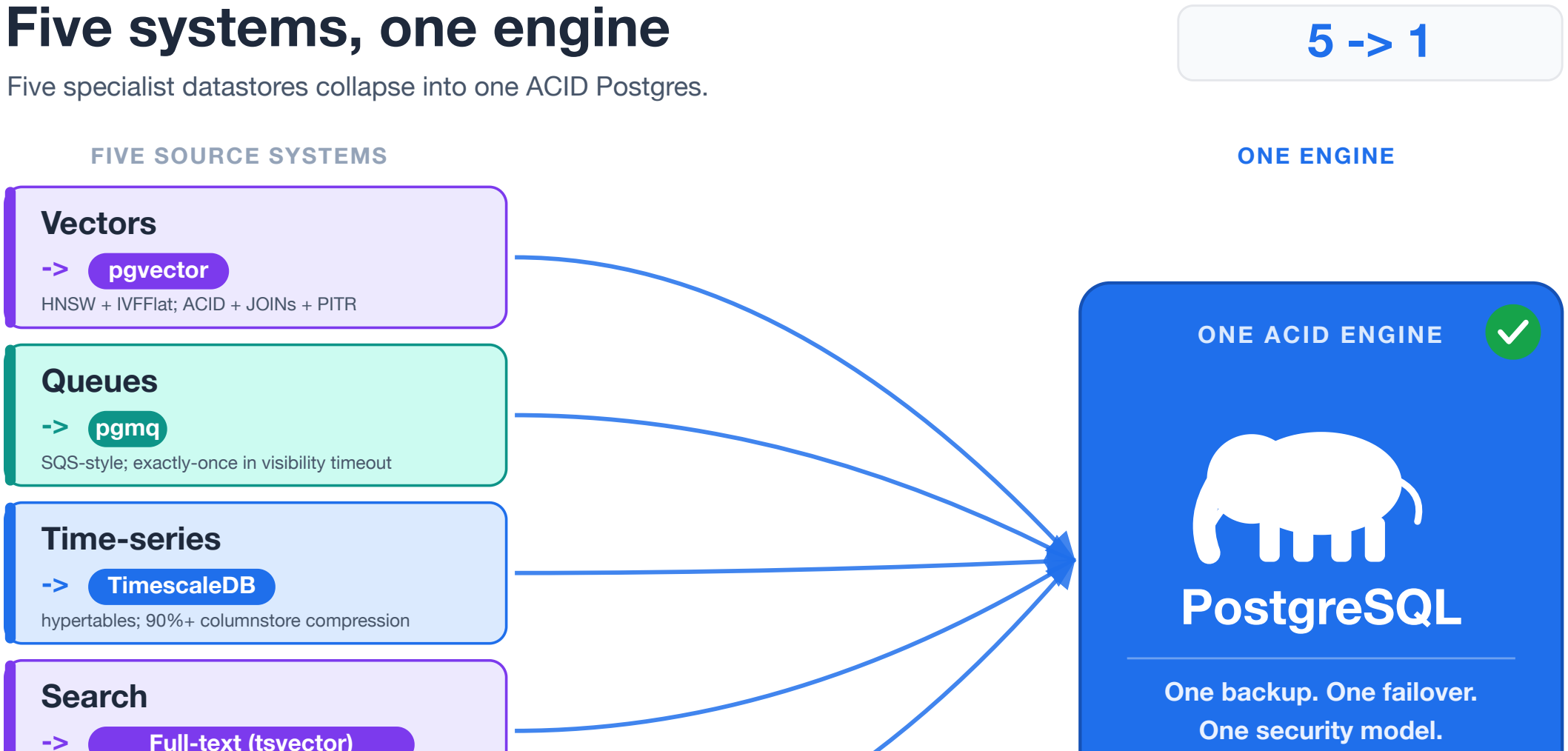
"Just use Postgres" stopped being a meme

It is not a half-joke about résumé-driven database sprawl anymore. It is the measured default.

- **48.7%** most-used database, two years running — up from **33% in 2018** *(Stack Overflow 2024)*
- **47.1%** most-admired **and 74.5%** most-desired — the one people use *and* want next
- **PostgreSQL 18** (Sept 2025) shipped async I/O for **up to 3× read throughput**; PG19 is already in beta

The engine is getting *better* at exactly the workloads that used to push you off it.

Five systems, one engine

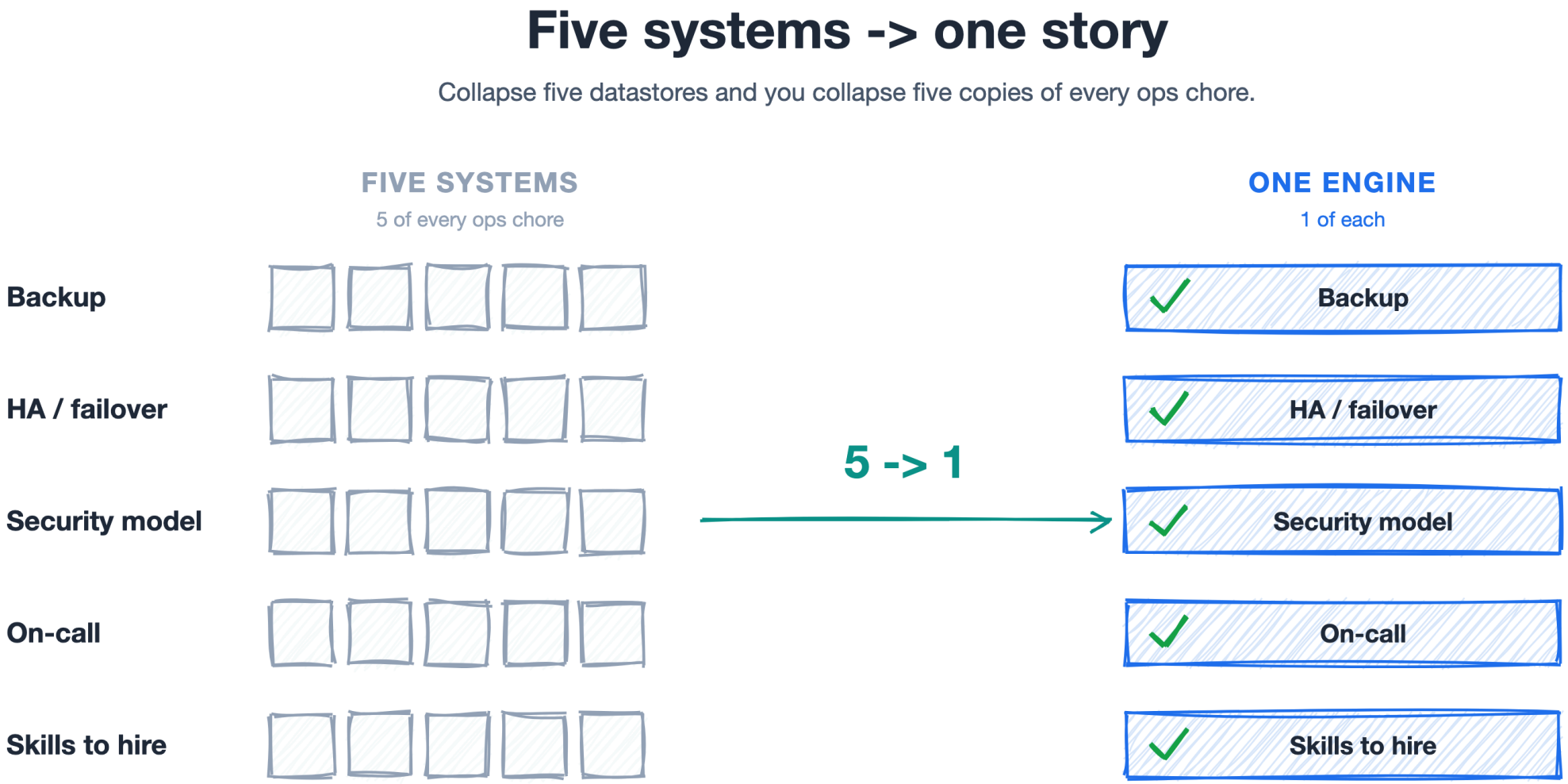


THE CONSOLIDATION CASE · ONE ENGINE, FIVE WORKLOADS

The extension does the job you were buying a database for

- **Vectors → pgvector** — similarity search *next to* your rows: HNSW/IVFFlat, plus ACID, JOINs, and point-in-time recovery. No second system to sync.
- **Queues → pgmq** — SQS-style queue where the **job and its data commit in the same transaction**. The "row saved but the job never fired" bug becomes impossible.
- **Time-series → TimescaleDB** — hypertables, continuous aggregates, a columnstore that routinely hits **90%+ compression**.
- **Search → `tsvector` + GIN** — real lexical ranking and phrase queries; fuse with pgvector for hybrid search.
- **Documents → JSONB** — a binary document type with GIN indexing — schemaless where you want it, relational where you need it.

Five systems → one ops story



The four walls

Where the belief stops -- the four walls

Four workloads where I leave Postgres on purpose -- each justified by one ceiling number.

1. Extreme write throughput

7.5M inserts/sec

@ 4 ms P99 (ScyllaDB, vendor; 2-5x Apache Cassandra)

Cassandra / ScyllaDB

the legitimate specialist exit

Why not PG: a single primary funnels all writes through one node.

Azure: Managed Cassandra / Cosmos DB

2. Planet-scale sharding

ALL of YouTube, 5+ yrs

Vitess ran YouTube's entire DB traffic (also Slack, Square, JD.com)

Spanner / CockroachDB / Vitess

the legitimate specialist exit

Why not PG: no transparent native sharding -- you bolt it on.

Azure: Cosmos DB / PostgreSQL Elastic Clusters

3. Sub-millisecond key-value

4. True OLAP at petabyte scale

THE DECISION RULE · ONE LINE

Name the wall, or you don't have one

Can't name the wall with a number? You haven't hit one -> just use Postgres.

The four walls: write throughput / planet-scale sharding / sub-ms key-value / petabyte OLAP.

Source: just-use-postgres.md (the decision rule); verified 2026-06-26.

A FRAMEWORK FOR DECIDING · FOR THE PEOPLE WHO CHOOSE

Default to Postgres, justify the exit

- **Make Postgres the default of record** — new workloads land there unless a named wall is proven. The burden of proof flips to whoever wants a new system.
- **Demand a number, not a vibe** — every proposal must name its wall and the threshold: write rate, shard count, latency floor, scan size.
- **Price the operational tax** — a specialist must beat Postgres by enough to pay for its own backup, failover, security, and on-call overhead.
- **Prove it cheaply** — spend a week consolidating one workload (pgvector / pgmq / TimescaleDB) and measure *before* you buy.

If no one can fill in the number, the wall is hypothetical — the default holds.

BEFORE YOU ADD THE SIXTH DATASTORE

Default to Postgres. Specialize on purpose.

Ask the only question that matters: **which of the four walls am I actually hitting, and what is the number?** If you can name it, specialize with confidence. If you can't — just use Postgres.

Run the audit this week: for every datastore beyond Postgres, can your team name the wall it clears and the number? The ones that can't are consolidation candidates. Then tell me the call you made — and which wall, if any, forced your hand.

Full consolidation case, the four breakpoints with their numbers, and the default-to-Postgres decision framework in the write-up.